AgentWorkShop: A Reference Implementation for Task-to-Device Integration in Industrial Agents

***Abstract*—Industrial agents need a software path that connects a task to production context, semantic operations, and configured device interfaces. AgentWorkShop is a modular reference implementation that composes task services, registered harnesses, line–product–recipe–run context, per-call operation checks, semantic data access, protocol drivers, and service-owned records. It resolves context for each agent turn and checks applicable authority at dispatch; prompt context grounds a proposal but is not a task-pinned authorization token. We evaluated one registered harness in cast-film and annealing simulators with different signal and protocol mappings. Six sequential tasks received synthetic approval and readback-verified receipts, each linked to a subsequent run-tagged probe; all six measurements remained outside their target bands. A deterministic fixture replay passed 31 assertions without an external language model. A focused mock-state ablation found that omitting only the post-approval context comparison allowed six stale product or run changes across three recipe routes to trigger mock-driver writes; other checks blocked the remaining six cases. These results validate the tested software paths and one check's incremental coverage, not process attainment, normal online data acquisition, cross-harness portability, or platform-level comparative benefit.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect each task to its production context, process signals, and equipment interfaces. Existing work addresses end-to-end automation control, operator-reviewed control-sequence generation, modular factory skills, and production scheduling [1]–[4]. Other studies target PLC code generation, context-aware control, resource capability discovery, shop-floor coordination, digital-twin fault handling, natural-language machine access, and robotic-line control [5]–[10]. Across production lines, agent roles, signal maps, protocols, objectives, and product/run context vary. An integration architecture must accommodate these differences around shared task and operation services while keeping each action tied to its active production context. Recent industrial-agent architectures add AAS-grounded OPC UA actions with signed approval, P&ID-grounded proposal checks, and governed sensor–MES–ERP evidence streams [11]–[13]. These works address asset-specific execution, process validation, or closed-loop analytics. This paper addresses the complementary software-integration question of composing task and harness routes, production context, semantic protocol I/O, and route-owned records.

Consider two production scenes with different process signals and interfaces: cast-film thickness control and annealing-zone temperature control. An agent task in either scene needs production context, typed signal access, and a route from a named operation to the configured device protocol. The framework must also distinguish a command receipt from a subsequent process observation. The shared task and operation services can therefore be configured with scene-specific signal maps, protocols, and limits while retaining a common execution path. The evaluation instantiates this path in two simulator scenes.

This paper presents AgentWorkShop as a reference implementation of task-to-device integration for industrial agents. It connects collaborative tasks and registered harnesses to production context, manufacturing execution system (MES) data, analysis and modeling services, data-acquisition (DAQ) and write-control (DCW) nodes, and protocol-specific drivers. Its interfaces include command-line, web, SDK, REST, Model Context Protocol (MCP), and Agent-to-Agent (A2A) surfaces. Task and chat routes enter the collaboration runtime, whereas a direct REST tool-invocation route can call the shared operation dispatcher without creating a task. Scene blueprints define signal maps, objectives, and guards; channel profiles, tool policies, and agent bindings scope operations. Agents and callers invoke named operations through shared runtime checks; the framework does not choose routes autonomously and retains proposals, command confirmation, and process observations as distinct records. Fig. 1 presents the system architecture, and Fig. 2 shows the cross-module task flow. Section IV details the node-exploration and recipe lifecycles.

This paper asks whether a configured registered-harness path can execute bounded tasks in two simulator scenes with different signal and protocol mappings while retaining a checkable relation from task through approval and verified route receipt to a subsequent run-tagged measurement. A deterministic fixture separately replays the software route without external model decisions. A 20-case mock-state suite injects line, product, recipe, and run changes during approval for `recipe_apply`, `recipe_trial`, and dispatching `recipe_rollback`; all stale actions are refused before mock-driver writes. In a focused test-only ablation that omits only the post-approval tuple comparison, six stale product/run changes reach mock actuation, while independent checks still refuse the other six cases.

The paper makes two contributions:

1) It specifies and implements a modular task-to-device reference path that joins collaborative tasks and registered harnesses to turn-time production context, per-call operation checks, semantic I/O, configured protocol drivers, and service-owned records. For approved recipe actions, the implementation binds the approval to a line–product–recipe–run tuple and refuses execution if that tuple changes while approval is pending. This distinguishes prompt grounding, dispatch authority, device receipt, and later process observation; it is an

implementation contribution, not a new protocol, safety-verification method, or process-control algorithm.

2) It evaluates selected paths in cast-film and annealing simulator scenes. In R20, six sequential tasks through one registered harness passed the task-to-approval-to-receipt-to-sample linkage checks; none of the six samples reached its target band. In R21, a deterministic fixture executed one task per scene with no external model and passed 31 runner assertions, including cleanup checks. A separate mock-state suite refused all 12 stale line–product–recipe–run mutations on three recipe actuation routes before mock-driver writes; a targeted ablation isolates the additional coverage for product and run changes provided by the tuple comparison.

Section II reviews related industrial-agent and integration work. Section III presents the design rationale, architecture, and cross-module execution flow; Section IV explains module responsibilities and their integrated operation. Section V describes the experiments and results, and Sections VI and VII discuss the evidence limits and conclude the paper.

II. Related Work

This section relates AgentWorkShop to plant-facing industrial agents and to production and automation abstractions for context, tool access, and operational constraints. Table I compares integration and execution features described in selected industrial-agent architectures, including systems with adjacent but distinct process-control objectives. The comparison is focused rather than exhaustive; a feature marked as not specified was not described in the cited study and is not inferred to be absent. Studies were included in this comparison when they describe an industrial agent's production context, plant-facing operations, or an action, approval, or evidence pathway. This criterion defines the table's scope; it is not a comprehensive survey or a performance ranking. Feature entries were coded from the cited study's full text only when the feature was explicitly described; otherwise the cell is marked "Not specified." Evaluation descriptions are transcribed at the level reported by each source, without normalizing sample units, success criteria, or metrics across studies.

A. Industrial Agents and Plant Integration

Industrial-agent systems, including large language model (LLM)-based systems, span end-to-end automation control, supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Xia et al. present an LLM agent framework for end-to-end industrial automation control, using structured prompts and event-driven context for production planning and operation [1]. Huang et al. present a multimodal architecture whose stated focus is automated control generation for industrial cyber-physical systems [2]. Biskupovic et al. study agent-based supervision for service-oriented industrial systems [14]; Agents4PLC and InstructMPC address PLC code generation and human–LLM context-aware control, respectively [5], [6]. Manufacturing studies consider dynamic resource-capability exploration [7], multi-agent shop-floor coordination [8], and digital-twin-supported fault handling [9]. Natural-language machine access [10] and real-time LLM control of a robot-operated serial line [4] extend this work to operator interfaces and robotic-line control. Together, these studies address different operational contexts, interfaces, and allocations of responsibility between agents, operators, and controllers.

The closest architecture-level comparison is Xia et al., who connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences; their case studies address robotic automation and process simulation [15]. Other nearby designs expose factory-module operations through MCP with state-based tool filtering, plan constrained manufacturing steps, or compose semantic recipes from OPC UA skills [3], [16], [17]. These studies describe important parts of plant-facing integration. The recent systems in Table I also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell and route approved actions through OPC UA Methods, with signed approvals and audit logging [11]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [12]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control, evaluated on real manufacturing records [13]. These works address asset-specific approval, process validation, or closed-loop analytics. This study examines a narrower implementation contract: a recipe action awaiting human approval carries its line–product–recipe–run context, rechecks that same tuple before actuation, and retains command receipt separately from a later process observation. The deterministic tests check refusal after line, product, recipe, or run changes during pending approval on three recipe actuation routes; R20 and R21 trace selected registered-harness paths across two simulator mappings. Table I reports described features; it does not establish that other systems lack equivalent mechanisms.

The operation checks complement safety-aware decision support [18] but do not replace independent process protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [19]; the operation-level scope, policy, grant, and binding checks discussed here are AgentWorkShop's own dispatcher rules, distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [20], [21]. Run-to-run methods connect process observations to successive production runs [22]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [23], [24]. Together, these works provide complementary abstractions for production context, tool interaction, and information flow. AgentWorkShop

TABLE I
FEATURE-LEVEL COMPARISON OF SELECTED PLANT-FACING INTEGRATION STUDIES; ENTRIES SUMMARIZE REPORTED DESIGNS AND EVALUATIONS, NOT PERFORMANCE.

System	Task and production context	Per-call operation checks	Device-facing route	Reported evaluation and record link
Xia et al. [15]	Cognitive, twin, and physical layers; snapshots, events, and plans	Not specified	Digital-twin/physical-automation integration	Robotic automation and process-simulation case studies
Köhle et al. [3]	Factory-module agents with injected live state	State-based tool filtering	Per-module MCP exposes OPC UA methods; MQTT coordination	Nine simulated production challenges
Vieira da Silva et al. [16]	LLM plans manufacturing steps under constraints	Not specified	MCP-exposed manufacturing resource functions	Laboratory multi-step process with constraint handling
Köhle et al. [17]	AI-assisted recipes composed from OPC UA skills	Not specified	OPC UA skills and virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [11]	AAS-grounded asset and production context	Signed HITL approval; immutable audit log	PLC-validated OPC UA Methods	Two laboratory PoCs; single scripted runs
Schall [12]	P&ID, SCADA, time-series, and SOP context	Deterministic topology and rule checks	Advisory proposal, gated execution	Five simulated scenarios; 43 injected proposals and 50 LLM runs
Lin and Chen [13]	Sensor–MES–ERP digital thread	SOP-, risk-, and capability-aware gating	Edge monitoring and closed-loop actions	25,275 real records; monitoring and operational metrics
This work	Collaborative tasks and direct calls; per-turn line–product–recipe–run context	Scope, policy, grant/binding at dispatch; route-specific state rechecked before write	Semantic node resolves to configured DAQ/DCW driver	R20: 6/6 live-agent traces; R21: 2/2 deterministic fixture traces across two mappings; no process attainment claim

composes such abstractions with its task runtime, operation checks, semantic I/O, and route-specific records; this paper evaluates selected paths through that composition rather than proposing a replacement for the underlying standards.

III. System Design

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. Its architecture organizes these variations around shared task and operation services, checks each proposed operation against the active production context before device routing, and retains route-specific evidence that distinguishes a command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements—scene variability, harness variability, operation authority, outcome observability, and extensibility—organize six design rules (D1–D6). These are deployment constraints drawn from the target workflows. D1 stores scene settings as versioned data; D2 provides registered harness, tool, and driver interfaces; D3 separates proposal from execution; D4 treats the tool catalog as discovery rather than permission; D5 routes actuator writes through the common DCW admission point; and D6 retains evidence by operation.

The requirements trace to implementation as follows. Scene variability maps to D1, scene blueprints, and context resolution; harness variability and extensibility map to D2, the registered harness and driver interfaces; operation authority maps to D3–D5, the shared dispatcher, grants, bindings, and write admission; outcome observability maps to D6, route-specific receipts, and observations. R13 exercises two scene-specific protocol and data paths; R20 exercises the configured task-to-device path through one registered harness, synthetic approvals, and route-specific records. Section V reports the experimental scope and evidence limits. AgentWorkShop supplies task and operation services; agent strategy and independent process protection remain outside this design.

B. Layered Architecture

Fig. 1 summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration band, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. Direct REST tool calls and the stdio MCP operation bridge can call the shared dispatcher without creating a task or invoking a harness; MCP management and task methods remain on their service routes. Direct calls have no agent-turn prompt context, so handlers resolve current route state from the request and services at invocation. The governance band projects an operation catalog and checks each invocation. The context band assembles bound-line and active-run information for prompt grounding, while operation handlers resolve the live state relevant to their route. The

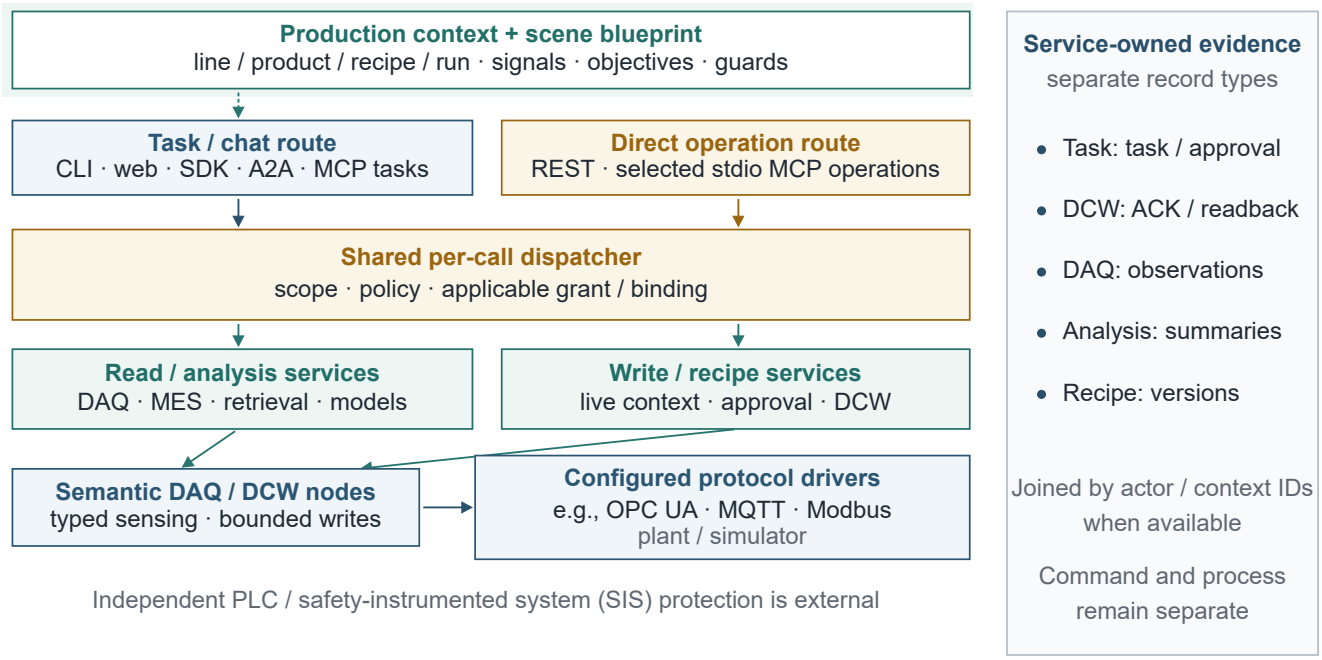


Fig. 1. AgentWorkShop architecture and record ownership. Task/chat calls pass through the listed access surfaces and a registered harness; direct REST and selected stdio-MCP operation calls enter the shared dispatcher, while MCP management/task methods use their service routes. Recipe writes recheck the approved line–product–recipe–run tuple; each service retains its own records. Independent programmable logic controller and safety-instrumented system protection remains external.

service band provides acquisition, write control, analysis, retrieval, and optional model services. The field-I/O band resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility. Task/chat routes enter the task runtime; direct operation routes enter the dispatcher; and both use shared operation checks where applicable. A harness adapter supplies agent-specific interaction on task paths, semantic nodes hide protocol details from operations, and the driver registry provides the device-facing extension point. Owning services retain task, command, observation, and version records separately and link them to available actor and production context. No single status conflates these outcomes.

C. Cross-Module Execution Flow

Fig. 2 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context for each agent turn. This context is grounding information, not a task-pinned transaction. For each returned tool call, the shared dispatcher rechecks invocation scope and channel policy and, for industrial operations, current grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a,c,u) = \bigwedge_{X \in \mathcal{C}(u)} X(a,c,u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, runtime grant, and agent binding/resource ownership, respectively. $\mathcal{C}(u) \subseteq \{S,P,G,B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is omitted only for an explicitly grant-exempt industrial tool, and B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. This condition is necessary for handler selection, not sufficient for actuation: the selected route applies its own guards and approval, then rechecks authority, context, and applicable limits before dispatch.

Fig. 3 gives reference pseudocode for the registered-harness task path, not one literal orchestrator function. Direct REST tool invocation bypasses the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

The architecture in Fig. 1 is implemented as cooperating modules rather than a single monolithic agent. Shared interfaces connect task execution to configuration, industrial services, protocol drivers, and route-specific evidence. This section describes those responsibilities and the checks that preserve their boundaries.

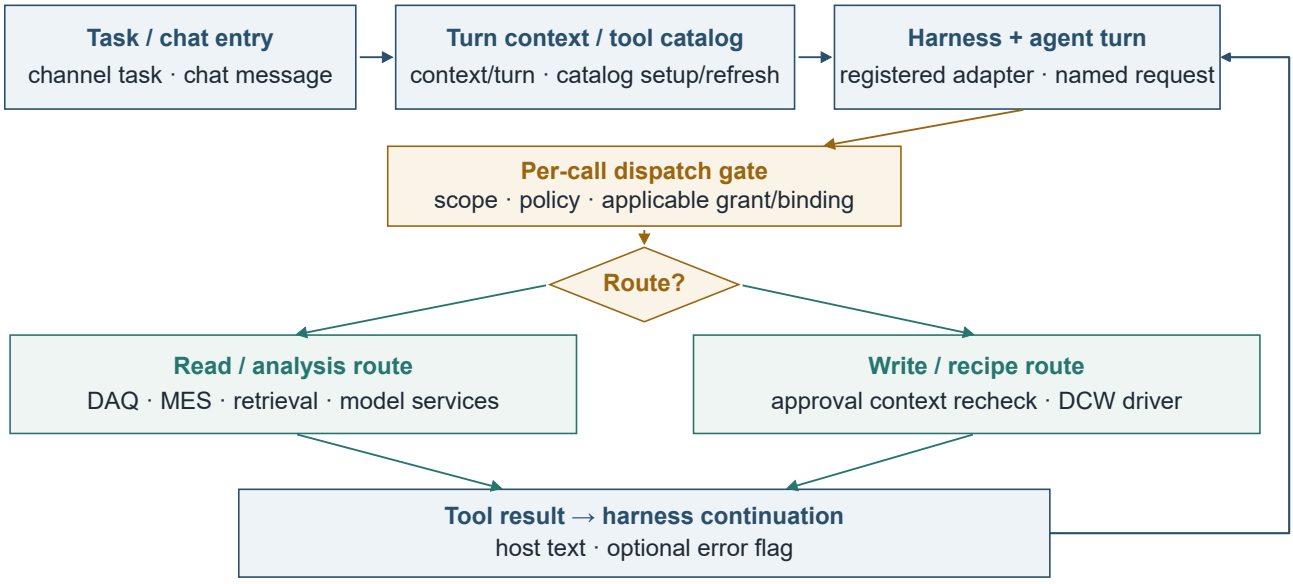


Fig. 2. Registered-harness task flow. Context is assembled per turn; the scoped tool catalog is loaded at setup or refresh. The dispatcher checks scope, policy, and applicable grants and bindings, then returns each tool result to the active harness turn. Direct REST and selected stdio-MCP operations enter the shared dispatcher without a task. In R20, the follow-up probe, ingest, and run-tagged query were initiated by the test runner outside the agent task. A command receipt remains distinct from a process observation.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service
$x_t \leftarrow \textsc{ResolvePromptContext}(c, q)$
$m \leftarrow \textsc{RetrieveMemory}(q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x_t, m)$
$h \leftarrow \textsc{InvokeRegisteredHarness}(a, p, T)$
for all tool request u returned by h **do**
  $g \leftarrow \textsc{CheckScopeAndChannelPolicy}(a, c, u)$
  if not g **then**
    EmitToolErrorToHarness(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow \textsc{CheckGrantBindingOwnership}(a, u)$
    if not v **then**
      EmitToolErrorToHarness(h,u); **continue**
    end if
  end if
  $k \leftarrow \textsc{SelectHandler}(u)$
  $r \leftarrow \textsc{ExecuteRouteHandler}(k, a, c, u)$ ▷ handler resolves live state and may persist route-specific records
  EmitDeclaredToolResultToHarness(h,r)
end for
return task state and tool results

Fig. 3. Reference pseudocode for the registered-harness task path. Direct operation invocation is not shown. Evidence persistence is route-specific and remains with the owning service; the shared tool result does not carry a generic evidence envelope.

The implementation scope is broader than the evaluated scope. Table II separates implemented interfaces from the evidence collected here. An interface intended for extension does not by itself establish interchangeability or portability.

The extension contracts expose the data passed at each boundary. A harness call is normalized as `HostToolCall(toolName, arguments)` and returns `HostToolResult(text, isError)`. A DCW driver accepts an engineering value, tolerance, range, and driver configuration; its result separates write status, raw value, readback, and acknowledgment grade (readback-verified, transport-ack, or unverified). Each DAQ sample stores its node, timestamp, value, and state, with optional line, product, recipe, and run IDs. A write anchor records line/node, prior and requested values, source, actor, time, and optional run/task/approval IDs; recipe definitions retain separate versions. These fields keep tool errors, device receipts, observations, and production lineage inspectable as separate records.

DAQ and write-anchor records may omit line, product, recipe, run, task, or approval identifiers when a route does not supply them. For the R20 trace audit, the join keys are required: task ID, approval ID, recipe-trial run ID, active-run ID, and the run ID returned with the sample must agree with the corresponding route records. R21 checks the corresponding IDs on its deterministic fixture path. Thus the reported complete joins are requirements of those evaluated records, not a guarantee that every general record contains every identifier.

A. Task Runtime and Harness Adapters

Task and chat access surfaces create or join a workspace channel, where the task engine manages members, task queues,

shared memory, and execution templates. Direct REST tool calls and selected stdio MCP operation calls can enter the shared dispatcher without creating a task or invoking a harness; MCP management and task methods use their corresponding services. A direct call has no agent-turn prompt context: its operation handler resolves live route state from request identifiers and service data at invocation. The harness registry instantiates an adapter behind a common agent interface; the prompt builder combines the task with the active scene brief, industrial context, channel profile, agent binding, and retrieved memory. Tool calls return to the shared dispatcher regardless of which registered harness produced them. The registry keeps engine-specific execution behind the common agent interface. Industrial context is assembled for each agent turn rather than pinned for the task lifetime, so prompt context is grounding information rather than write authorization. An approved recipe action separately captures the current line–product–recipe–run tuple when approval is requested and compares it again before dispatch. This approval-time check does not imply that the earlier prompt context remained unchanged.

B. Production Context and Configuration

Scene blueprints describe devices, signal maps, objectives, and process guards. A task resolves these definitions against its active line–product–recipe–run context. The channel profile selects the available capability family, the tool policy narrows the operation catalog, and the agent binding scopes the nodes, controls, and recipe operations available to that member. These filters can restrict one another but cannot broaden the underlying device range. The dispatcher re-reads policy at execution, so a cached catalog does not retain authority after a grant or policy change. The prompt builder assembles the agent's available bound-line and active-run context on each turn; it does not pin the complete context tuple to the task. Approved recipe actions capture the active tuple at request time and refuse dispatch if its line, product, recipe, or run field changes during the approval wait. Specific production snapshot, trial, and MPC routes, together with Hybrid Twin training submissions, resolve and validate a versioned `SceneContract` at operation time. The Twin physics-specification draft route can instead accept a caller-supplied scene description; SceneContract enforcement is therefore route-specific and does not validate or pin all prompt context.

C. Acquisition, Analysis, and Modeling Services

Semantic DAQ nodes expose typed values, engineering units, ranges, and sampling settings while hiding protocol-specific reads. A per-node DAQ runtime schedules acquisition and stores timestamped observations; the gateway aggregates node pipelines. MES-facing operations provide the configured production-data path. Analysis and retrieval services consume these records, while optional model, training, optimization, and digital-twin services are invoked through their registered operation handlers. These services complement the agent workflow; AgentWorkShop does not prescribe a single optimization strategy.

D. Semantic I/O and Actuation

The driver registry resolves a semantic node to a built-in or plugin driver. DAQ handlers use the node's acquisition interface; DCW handlers route a write or readback through the controller and the configured protocol driver. This separation lets recipes and agents refer to semantic controls without embedding Modbus, OPC UA, MQTT, HTTP, or MES transport details in their operation arguments. The supported driver mappings are implementation interfaces; only the mappings and workflows enumerated in Section V are experimentally exercised.

E. Shared Dispatch and Write Admission

Every named operation passes the dispatcher. Its ordered checks validate invocation scope and current channel policy (tool selection, profile, and applicable duty separation), then the agent's binding and runtime grant and ownership of referenced resources. A refusal returns the excluding condition. Denial persistence is selective: grant denials and selected resource-reference denials have explicit audit paths, whereas scope and channel-policy denials are not uniformly retained as durable audit events. A catalog entry therefore describes availability but does not authorize execution. Handlers for operations that require manual approval suspend for a decision, then revalidate authority and active limits before dispatch. For manual recipe operations, the handler checks the active run, candidate parameters, and route timing before approval; after the wait it rechecks current authority and active limits. For `recipe_apply`, a changed definition requires a new proposal. These route-specific checks complement, but do not replace, the per-call dispatcher checks.

All actuator writes converge on the DCW admission point. For an online node write, the admissible interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits. Missing optional limits impose no restriction, whereas an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch applies the hard, baseline, and product limits to the replacement recipe rather than intersecting with the previous recipe window. The actuator path also enforces step, interval, and hold-window checks. These admission rules constrain commands; process objectives and coupled variable guards are assessed from later observations. The framework does not replace independent PLC or safety-instrumented protection.

F. Integrated Operation and Route-Specific Evidence

A recipe trial illustrates how the modules cooperate. The task runtime delivers a named `recipe_trial` call from the agent harness; the dispatcher checks the agent's recipe binding and execution scope; the recipe handler checks the active run, candidate parameters, and route timing; and a manual binding requests approval before the DCW controller sends the batch through its driver. The handler returns a per-parameter receipt.

TABLE II
IMPLEMENTED INTERFACES AND EVIDENCE COVERAGE. "NOT EVALUATED" MARKS FEATURES OUTSIDE THIS STUDY.

Path	Implemented interface and checks	Evidence in this study
Task and harness	Registered harness adapters and task/chat services; direct REST operation calls share the dispatcher but bypass the task runtime.	R20 exercises six registered-harness tasks; R21 exercises one deterministic fixture task per scene. Alternate harnesses and direct REST calls are not evaluated.
Dispatch and write	Every call checks scope, policy, grant, binding, and resource ownership as applicable. Recipe operations capture line/product/recipe/run context at approval and recheck it before execution; changed context refuses the action. Candidate bounds, timing, recipe definition, and active limits are also checked.	R20/R21 exercise approved recipe trials and verified readback. Mock tests inject four stale-context mutations on each of recipe apply, recipe trial, and dispatching rollback. Online node writes are not evaluated end to end.
Semantic I/O	DAQ/DCW nodes resolve configured semantic signals to built-in or plugin drivers.	R20/R21 exercise two scene mappings; R13 provides exploratory protocol probes. The normal online DAQ backend and physical equipment are not evaluated.
Records	Owning services retain task, approval, receipt, observation, and recipe-version records separately; production identifiers are route-dependent.	R20 reports six complete task-to-sample joins; R21 reports two fixture-path joins. Neither result establishes universal record completeness across untested routes.

The caller then obtains DAQ observations and can request a separate update, apply, or rollback operation. A trial receipt confirms command handling; it does not by itself establish process attainment or persist a new recipe version.

The routes retain different records because they answer different questions. A driver acknowledgment reports command evidence; DAQ reports a process observation; an exploration summary records a bounded probe; and a recipe version records a saved definition. Their relation through actor and production context allows an execution to be inspected without treating these outcomes as interchangeable.

G. Route-Specific Lifecycles and Evidence

a) Timing and approval: Timing guards are route-specific. Online node writes enforce an effective minimum spacing of 60 s. Recipe dispatch uses a per-recipe interval anchored to approval, while trials also use a configurable per-line cadence (default 300 s). The recipe interval can refuse a proposal before an approval request is created. Checks occur at proposal or execution according to the operation, and experiment-specific pacing overrides are reported with the corresponding results.

Manual bindings create an approval request containing the actor, operation, candidate values, applicable limits, expiry, parameter bases, and the active line–product–recipe–run context. The default wait expires after 180 s and refuses execution. After approval, the executor rechecks authorization and compares the current context tuple with the approved tuple; a changed tuple is refused before actuation. A focused mock-state suite changes line, product, recipe, or run during approval for recipe apply, trial, and dispatching rollback; each mutation is refused before a driver write or new execution record. For `recipe_apply`, a changed recipe definition also requires a new proposal; an unchanged approval dispatches a copy of the approved parameter snapshot. A line-level recipe mode defaults to manual; changing it to automatic requires administrator confirmation. The approval record authorizes a workflow action and does not replace process protection.

Fig. 4(a) illustrates the four-layer interval in Eq. (2) with an archived RPM configuration; panel (b) shows a symbolic approval followed by context and authority rechecks. The plotted values are illustrative and are not R20 measurements.

b) Node probe: The bounded `optimization_explore` route is exposed to the AML optimization profile. It validates a bound control and DAQ scope, captures a baseline, routes a bounded write through DCW, and stores a later target window as a separate exploration summary. Its timing and approval behavior is route-specific; R20 does not exercise this route.

c) Recipe operations: `recipe_trial` checks the bound recipe and active run, then dispatches candidate overrides as a batch with per-parameter verification. The caller subsequently queries DAQ and may save a definition with `recipe_update`, restore one with `recipe_rollback`, or apply a version with `recipe_apply`. A trial does not save or apply a version, and endpoint verification is not a process-response measurement. Public direct-node control is closed; bounded probes and recipe operations are the configured write surfaces.

d) Command verification: Command confirmation and process response answer different questions. A driver ACK is graded as `readback-verified`, `transport-ack`, or `unverified`; a matching endpoint read supports the first grade, whereas transport or MES application confirmation supports the second. Recipe-batch verification is `verified`, `unverified`, or `failed`. If readback is absent or mismatched, up to three reads are attempted 2 s apart. Initial ACKs, final ACKs, and per-parameter verdicts are retained separately. A readback-verified setpoint does not establish target attainment.

e) Observation, verdict, and compensation: Recipe-path observation follows batch dispatch, and saving a definition, applying it, and observing process response remain separate operations. An open node record can request rollback after repeated breaches; missing samples do not count as breaches. Caller verdicts and compensation remain separate from the observation and subject to admission checks.

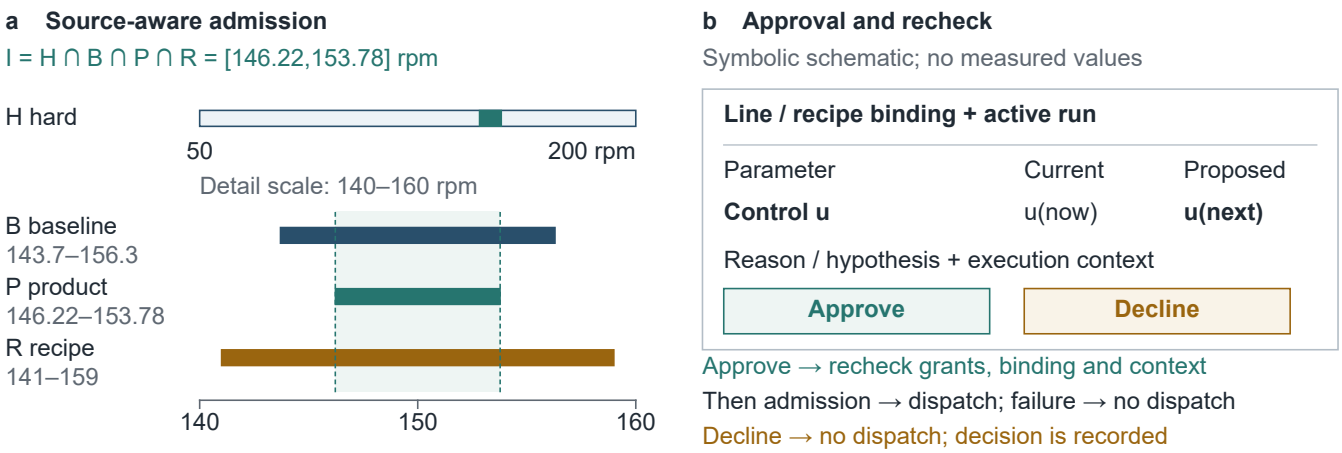


Fig. 4. Admission and approval boundaries. (a) Illustrative archived RPM configuration (not part of R20): online node interventions use the four-layer value interval of Eq. (2), whereas recipe dispatch omits the prior recipe window. (b) Symbolic human approval binds a proposed action and its context tuple; execution rechecks the active line–product–recipe–run context, authority, and applicable limits after approval. Approval does not reserve equipment state.

V. Experimental Evaluation

The evaluation asks whether one registered-harness path can execute bounded tasks across two simulator scenes with different signal and protocol mappings while retaining a checkable task-to-approval-to-receipt-to-sample trace. R20 is the primary result: six sequential tasks through one registered harness, three per scene on one simulator instance per scene. It demonstrates this configured software path, not configuration-reuse benefit or process attainment. R21 is a separate deterministic fixture replay on an isolated current-version application and simulator; it checks the same task-to-device path without an LLM decision. R13 is secondary exploratory protocol and data-path evidence whose exact runtime source was not pinned. All runs use repository simulators. Supplementary File S1 contains the R20 trace matrix and records, plus the R21 fixture evidence.

A. Evidence Units and Experimental Procedure

R20 uses the registered harness adapter named `omp` in cast-film and annealing scenes (`omp` is the project adapter key, not a network protocol), with three sequential task executions per scene on one simulator instance per scene. Each execution creates a new logical line, product, recipe, active run, channel, agent, and task on that shared scene simulator. Line start reapplies the recipe baseline (120 rpm or 730 °C), so the control read returns to baseline while the simulated process state carries forward. These are sequential software runs, not independent process replications. R13 uses configured protocol and data paths in the same two simulator scenes. Earlier experiments are retained separately in Supplementary File S1 and are not pooled with these results.

B. Registered-Harness Process Path (R20)

R20 ran the local v0.8.0 working-tree application with the registered `omp` adapter and GLM-5.3-Flash. Supplementary File S1 contains the source-reconstruction kit and lists its external prerequisites: the base repositories, OMP CLI, local model-service credentials, Node.js toolchain, and simulator runtime. Credentials are not included, and the external model service does not guarantee deterministic replay. Cast film used OPC UA for screw-speed control and MQTT for thickness; annealing used Modbus TCP for both the zone-3 setpoint and strip-temperature probe. The model received a live control read, process measurement, and target band, then selected one value from the scenario-defined prevalidated set (125/130/135 rpm or 735/740/745 °C) for a bounded `recipe_trial`; this was a constrained choice, not an unrestricted optimization search. A synthetic administrator approved each proposal. Each route log contains one trial and a 1/1 verified device receipt. Workspace history can mirror host-tool events; trial counts therefore use the route-specific recipe and acknowledgment records.

After verifying the route ACK and returned trial result, the runner initiated a 20-s delay, then probed, ingested, and queried one value per task under the active-run tag through the isolated SQLite time-series fallback. The runner checked task state after the sample query; all six task records were then `COMPLETED`. The runner reported 76 passing assertions and no failures, and the separate trace audit found a complete task-to-sample record in all six executions: each had a matching approval ID, one readback-verified trial receipt, and one query result whose run ID and value matched the active run and explicit probe. The task-completion timestamp was not retained, so no task-completion-to-sample interval can be computed. Table III labels Film-1–3 map to `cast-film-r1--r3`, and Anneal-1–3 to `annealing-r1--r3`; full identifiers and per-record checks are listed in `R20-TRACE-MATRIX.csv` in S1.

The trace matrix distinguishes two run identifiers. The trial run ID denotes the `recipe_trial` operation record; the active run ID denotes the production run used to tag and query the probe. Both are linked to the same task and approval record, and they are not expected to be equal.

Table III reports the pre-task process value, selected setting, and run-tagged sample for each execution; all six remained outside target. The results verify a bounded harness-to-simulator path, not process attainment or convergence. One probe is not a response trajectory, the SQLite fallback does not

TABLE III
BOUNDED RECIPE-TRIAL SETTINGS AND POST-READBACK SIMULATOR PROBES. ALL SIX SAMPLES REMAINED OUTSIDE THEIR TARGET BANDS.

Case	Pre-task value / target band	Control: start→trial	Sample
Film-1	55.82/48–52 μm	120 → 125 rpm	45.69 μm
Film-2	45.69/48–52 μm	120 → 130 rpm	46.69 μm
Film-3	46.69/48–52 μm	120 → 125 rpm	45.69 μm
Anneal-1	675.40/700–760 °C	730 → 745 °C	679.60 °C
Anneal-2	678.30/700–760 °C	730 → 745 °C	677.70 °C
Anneal-3	676.90/700–760 °C	730 → 745 °C	678.10 °C

test normal online DAQ, and R20 does not test profile/binding reuse.

C. Deterministic Fixture Replay (R21)

R21 ran the local v0.8.0 application with a deterministic transport fixture for the registered `omp` harness, which returned a fixed tool call without invoking an LLM or autonomous harness decision. In one fresh isolated application/simulator environment, one fixture task per scene invoked the recipe-trial path in cast-film and annealing. The runner checked dispatch, synthetic approval, verified receipt, matching device readback, run-tagged DAQ probe, and route log; all 31 runner assertions, including process and port cleanup checks, passed. This replay shows that the configured software route can be exercised with deterministic harness input. It is not an independent real-agent run, a process replication, or evidence of target attainment. The two fixture tasks are not pooled with R20.

D. Software Contract Checks

The local unit suite passed 566/566 tests across seven suites. Relevant contract checks refuse denied or expired approvals, changed recipe or limit state, and invalid node values before driver actuation; unchanged approved state dispatches through the mock driver and readback. The approval-snapshot suite passed 20/20 cases. Twelve deterministic fault injections changed one approval-time field at a time (line, product, recipe, or active run) on `recipe_apply`, `recipe_trial`, and `recipe_rollback(dispatch=true)`. With the production handler, all 12 stale actions were refused before a mock-driver write or new execution record. A focused test-only variant removed only the post-approval tuple comparison while retaining the other rechecks: the six product/run mutations (two fields across three routes) proceeded, each producing two mock-driver writes and one execution record; the six line/recipe mutations remained blocked by independent checks. This component ablation isolates the tuple comparison's incremental coverage for product and active-run identity. It is not a comparison with another platform. These tests use temporary or mock state and are separate from R20's 76 runner checks; they do not represent process trials or test missing/mismatched DAQ run tags in the end-to-end path.

E. Exploratory Protocol and Data Path (R13)

The R13 application report describes five protocol-family read probes (OPC UA, MQTT, Modbus TCP, Modbus RTU over TCP, and HTTP/REST) and bounded writes with matching readbacks in two simulator scenes. Six sequential 120-s windows per scene at 1-s sampling produced lineage-verified datasets with 658 cast-film rows and 655 annealing rows. The rows are overlapping time-window samples, not independent process replications. The exact runtime source was not recorded. R13 is used only as exploratory protocol and data-lineage evidence; its archived gate outputs and run-boundary audit are documented in the reproducibility supplement and do not support the primary registered-harness result. R14 is an earlier exploratory pilot with alternative signal pairs; both pilot model gates failed, and those outcomes informed the subsequent R13 pair selection. R13 also produced no eligible model, recommendation, or dispatch. The two runs remain separate and are not model-performance evidence.

Earlier protocol, admission-ablation, controller, deployment, and scripted recipe records in S1 use older versions or interfaces. They are not pooled with R13 or R20, used as matched strategy comparisons, or relied on for the current integration claims.

VI. DISCUSSION

R20 produced six complete task-to-approval-to-receipt-to-sample traces across two simulator scenes; none of its single post-readback samples reached target. R21 passed 31 assertions on the same configured route, and a 20-case mock suite refused twelve stale approval-context mutations across three recipe actuation routes before actuation. A test-only ablation that removed the post-approval tuple comparison allowed the product and active-run mutations to proceed on all three routes, while independent checks still blocked line and recipe changes. This isolates the added coverage of that comparison for those two context fields; it is not a baseline comparison of AgentWorkShop against another integration framework.

Each scene used one simulator instance and a single post-readback probe per task; the results therefore do not establish independent process replication, convergence, or quality improvement. Configuration reuse, alternate harnesses or direct REST entry, online DAQ, physical transfer, and comparative architecture value remain untested. Synthetic approval does not measure operator effectiveness [25]–[27]; tool restrictions do not establish deployment security [28]–[30] or replace independently engineered PLC/SIS protection [31]. Future work should compare a defined platform-level baseline and include independent simulator and physical-plant trials.

VII. CONCLUSION

AgentWorkShop connects collaborative tasks to production context, checked operations, semantic I/O, and service-owned records. R20 and R21 verify selected paths across two simulator mappings; a mock-state suite confirms refusal when an approved line–product–recipe–run tuple changes on three actuation routes. A targeted ablation shows that the tuple comparison adds mock-state refusal coverage for product and active-run changes beyond the remaining route checks. These results validate selected software behavior and leave platform-level comparative integration value and process performance for future evaluation.

AI Assistance Disclosure

OpenAI ChatGPT revised Sections I–VI and drafted Python for Figs. 1, 2, and 4. OpenAI image generation produced early concepts for Figs. 1 and 2; final vectors were redrawn and implementation-checked. AI generated no experimental records or measurements. The authors verified all submitted content.

References

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[12] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[13] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[14] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[15] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[16] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[17] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[18] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[19] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[20] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[21] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[22] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[23] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[24] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[25] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[26] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[27] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[29] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[30] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[31] ——, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.